

```
#include<stdio.h>
#include<math.h>
#define STACKSIZE 100
#define TRUE 1
#define FALSE 0
struct stack
{
int top;
char item[STACKSIZE];
};

struct stack s;
char infix[100];
char postfix[100];
int pos=0;
void push(char);
char pop();
void convert();
int empty();
int precedence(char);
void main()
{
s.top=-1;
printf("enter infix string \n");
scanf("%s",infix);
convert();
printf("The postfix expression:\n");
printf("%s", postfix);
}
```

```

void convert()
{
int i;
char symb, temp;
for(i=0;infix[i]!='\0';i++)
{
symb=infix[i];
switch(symb)
{
case '(': push(symb);
break;
case ')': while((temp=pop())!='(')
{
postfix[pos++]=temp;
}
break;

case '+':
case '-':
case '*':
case '/':
case '$': while(!empty()
&&
(precedence(s.item[s.top])>=precedence
(symb)))
{
temp=pop();
postfix[pos++]=temp;
}
push(symb);
break;

```

```
default: postfix[pos++]=symb;  
        break;
```

```
    }
```

```
}
```

```
while (!empty())
```

```
{
```

```
temp=pop();
```

```
postfix[pos++]=temp;
```

```
}
```

```
}
```

```
void push(char x)
```

```
{
```

```
if (s.top==STACKSIZE-1)
```

```
{
```

```
printf("stack overflow\n");
```

```
}
```

```
else
```

```
{s.item[++s.top]=x;
```

```
}
```

```
}
```

```
char pop()
```

```
{
```

```
if (empty())
```

```
{
```

```
printf("stack underflow\n");  
}  
return(s.item[s.top--]);  
}
```

```
int empty()  
{  
if (s.top==-1)  
return TRUE;  
else  
return FALSE;  
}
```

```
int precedence(char symb)  
{  
switch(symb)  
{  
case '$' : return(3);  
case '*' :  
case '/' : return(2);  
case '+' :  
case '-' : return(1);  
case ')' :  
case '(' : return(0);  
}  
}
```